

Tugas 6: Praktikum Mandiri 6 Machine Learning

Hisyam Wildan Alfath - 0110222206

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: hisy22206ti@student.nurulfikri.ac.id

Abstract. Praktikum ini bertujuan untuk mengklasifikasikan jenis obat menggunakan algoritma machine learning dengan dataset Drug200. Proses analisis meliputi tahap pra-pemrosesan data, encoding fitur kategorikal, pembuatan visualisasi, serta pembangunan dan evaluasi model klasifikasi SVM. Hasil evaluasi menunjukkan akurasi prediksi yang sangat tinggi, serta visualisasi data dua dan tiga dimensi memperjelas pemisahan masing-masing tipe obat. Praktikum ini membuktikan bahwa pemrosesan dan analisis data yang tepat sangat mendukung keberhasilan model klasifikasi obat secara efektif dan akurat.

1. Praktikum Mandiri 06

- Menghubungkan Colab dengan Google Drive

```
[1]
✓ 20s # Menghubungkan colab dengan google drive
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive
```

Kode ini memanggil modul drive dari google.colab untuk menghubungkan (mount) Google Drive ke lingkungan Google Colab, dan outputnya adalah muncul pesan “Mounted at /content/drive” yang menandakan folder Google Drive sekarang sudah bisa diakses pada path tersebut di Colab.

- Membaca dan Menampilkan Data CSV

```
[16]
✓ 0s import pandas as pd

# Membaca file csv menggunakan pandas
df = pd.read_csv("/content/drive/MyDrive/Hisyam Wildan Alfath/Semester 7/Machine Learning/Pertemuan 6/Praktikum & Tugas Mandiri 06/data/drug200.csv")

# Cetak header data (5 baris data) dari file
df.head()
```

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	F	HIGH	HIGH	25.355	DrugY
1	47	M	LOW	HIGH	13.093	drugC
2	47	M	LOW	HIGH	10.114	drugC
3	28	F	NORMAL	HIGH	7.798	drugX
4	61	F	LOW	HIGH	18.043	DrugY

Kode pada gambar digunakan untuk membaca file data berformat CSV menggunakan library pandas, kemudian menampilkan 5 baris data pertama dari file

tersebut dengan fungsi `df.head()`. Hal ini bertujuan untuk memastikan bahwa data berhasil dimuat ke dalam DataFrame dan untuk melihat struktur serta contoh isi data. Output yang dihasilkan menampilkan lima baris pertama dengan kolom-kolom seperti Age, Sex, BP, Cholesterol, Na_to_K, dan Drug, yang menunjukkan data masing-masing pasien dan jenis obat yang diberikan. Dengan cara ini, pengguna dapat melakukan pengecekan awal sebelum melanjutkan ke tahap analisis data selanjutnya.

-

```
[17]
✓ Os df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Age         200 non-null   int64
1   Sex         200 non-null   object
2   BP          200 non-null   object
3   Cholesterol 200 non-null   object
4   Na_to_K     200 non-null   float64
5   Drug        200 non-null   object
dtypes: float64(1), int64(1), object(4)
memory usage: 9.5+ KB
```

Kode ini digunakan untuk menampilkan informasi detail tentang DataFrame dengan fungsi `df.info()`. Output yang dihasilkan menunjukkan jumlah baris data (200 entries), nama dan jumlah kolom (6 kolom), tipe data setiap kolom (int64, float64, object), serta jumlah data non-null pada masing-masing kolom. Semua kolom memiliki 200 nilai non-null, yang berarti tidak ada data yang hilang (missing value), dan tipe data seperti Age bertipe int64, Na_to_K bertipe float64, sedangkan kolom lain seperti Sex, BP, Cholesterol, dan Drug bertipe object. Informasi ini penting untuk memastikan kelengkapan data sebelum melakukan analisis lebih lanjut.

- Statistik Deskriptif Data Numerik

```
[18]
✓ Os df.describe()
```

	Age	Na_to_K
count	200.000000	200.000000
mean	44.315000	16.084485
std	16.544315	7.223956
min	15.000000	6.269000
25%	31.000000	10.445500
50%	45.000000	13.936500
75%	58.000000	19.380000
max	74.000000	38.247000

Kode ini digunakan untuk menampilkan statistik deskriptif dari data numerik pada DataFrame menggunakan fungsi `df.describe()`. Output yang dihasilkan memberikan informasi statistik seperti jumlah data (count), nilai rata-rata (mean), standar deviasi (std), nilai minimum (min), kuartil pertama (25%), median (50%), kuartil ketiga (75%), dan nilai maksimum (max) untuk kolom Age dan Na_to_K. Statistik ini sangat membantu untuk memahami sebaran dan karakteristik nilai-nilai numerik dalam dataset secara cepat sebelum melakukan analisis lebih lanjut.

- Menampilkan Nilai Unik dan Distribusi Data Kategorik

```
[19]
✓ Os df["Drug"].unique()

array(['DrugY', 'drugC', 'drugX', 'drugA', 'drugB'], dtype=object)

[20]
✓ Os df["Drug"].value_counts()

count
Drug
DrugY    91
drugX    54
drugA    23
drugC    16
drugB    16
dtype: int64
```

Kode ini digunakan untuk mengecek nilai unik dan menghitung distribusi setiap kategori pada kolom "Drug". Fungsi `df["Drug"].unique()` menampilkan seluruh kategori obat yang ada dalam data, yaitu DrugY, drugC, drugX, drugA, dan drugB. Selanjutnya, `df["Drug"].value_counts()` digunakan untuk menghitung jumlah kemunculan masing-masing jenis obat, di mana hasilnya menunjukkan DrugY sebanyak 91, drugX sebanyak 54, drugA sebanyak 23, serta drugC dan drugB masing-

masing sebanyak 16 kali. Analisis ini sangat membantu untuk memahami proporsi data dalam variabel kategorik sebelum melakukan analisis lebih lanjut.

- Cek Data Duplikat

```
[21] ✓ Os
# Cek duplicate
df.duplicated().sum()

np.int64(0)
```

Kode ini digunakan untuk memeriksa apakah terdapat baris data yang duplikat di dalam DataFrame dengan fungsi `df.duplicated().sum()`. Output yang dihasilkan adalah 0, yang artinya tidak ada baris data yang duplikat pada dataset ini. Dengan demikian, seluruh data bersifat unik dan dapat digunakan langsung tanpa perlu membersihkan duplikasi terlebih dahulu.

- Memisahkan Fitur dan Target

```
[22] ✓ Os
# Fitur (X) tanpa kolom SkinThickness dan BloodPressure
X = df[['Age', 'Sex', 'BP', 'Cholesterol', 'Na_to_K']]

# Kolom target (y)
y = df['Drug']
```

```
[23] ✓ Os
X.head()
```

	Age	Sex	BP	Cholesterol	Na_to_K
0	23	F	HIGH	HIGH	25.355
1	47	M	LOW	HIGH	13.093
2	47	M	LOW	HIGH	10.114
3	28	F	NORMAL	HIGH	7.798
4	61	F	LOW	HIGH	18.043

Next steps: [Generate code with X](#) [New interactive sheet](#)

```
[24] ✓ Os
y.head()
```

	Drug
0	DrugY
1	drugC
2	drugC
3	drugX
4	DrugY

dtype: object

Kode ini digunakan untuk memisahkan data menjadi dua bagian utama: fitur (X) dan target (y). Fitur (X) terdiri dari kolom Age, Sex, BP, Cholesterol, dan Na_to_K, yaitu variabel-variabel yang akan digunakan sebagai input dalam proses analisis atau

pemodelan. Sementara kolom target (y) berisi label Drug, yaitu jenis obat yang menjadi output yang ingin diprediksi. Output dari X.head() menampilkan lima baris pertama dari data fitur, sedangkan y.head() menampilkan lima baris pertama dari data target. Pemisahan ini merupakan langkah penting sebelum melakukan pelatihan model machine learning.

- Mengubah Data Kategorikal ke Numerik

```
[25] ✓ 0s
# fitur kategorikal lain (Sex, BP, Cholesterol)
for col in ['Sex', 'BP', 'Cholesterol', 'Drug']:
    if col in df.columns:
        df[col] = df[col].astype('category').cat.codes

df.head()
```

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	0	0	0	25.355	0
1	47	1	1	0	13.093	3
2	47	1	1	0	10.114	3
3	28	0	2	0	7.798	4
4	61	0	1	0	18.043	0

Kode ini digunakan untuk mengonversi kolom-kolom kategorikal ('Sex', 'BP', 'Cholesterol', 'Drug') menjadi nilai numerik dengan menggunakan kategori kode. Proses ini dilakukan dengan mengubah setiap kolom menjadi tipe 'category' lalu mengambil atribut cat.codes sehingga setiap kategori pada kolom tersebut diwakili oleh angka. Output df.head() menunjukkan bahwa nilai pada kolom-kolom tersebut kini telah berubah ke bentuk numerik, misalnya Sex, BP, Cholesterol, dan Drug yang semula berupa teks kini tampil sebagai 0, 1, 2, atau 3. Transformasi ini penting agar data dapat digunakan dalam algoritma machine learning yang hanya mendukung data berformat numerik.

- Encoding Data Kategorikal dengan LabelEncoder

```
[33] ✓ Os from sklearn.preprocessing import LabelEncoder

# Membuat instance dari objek LabelEncoder
encoder = LabelEncoder()

# Melakukan iterasi pada setiap kolom non-numerik (object) di fitur X
# dan mengubahnya menjadi nilai numerik.
for col in X.select_dtypes(include='object').columns:
    X[col] = encoder.fit_transform(X[col])

# Mengubah label target (y) menjadi format numerik
y = encoder.fit_transform(y)

# Menampilkan lima baris pertama dari data fitur untuk verifikasi hasil
print("Setelah Encoding:")
display(X.head())
```

Setelah Encoding:

	Age	Sex	BP	Cholesterol	Na_to_K
0	23	0	0	0	25.355
1	47	1	1	0	13.093
2	47	1	1	0	10.114
3	28	0	2	0	7.798
4	61	0	1	0	18.043

Kode ini digunakan untuk mengubah data kategorikal yang bertipe object menjadi numerik dengan bantuan LabelEncoder dari library scikit-learn. Langkah pertama adalah membuat instance LabelEncoder, lalu melakukan iterasi pada setiap kolom bertipe object di fitur X dan mengonversinya ke format numerik. Setelah itu, label target (y) juga diubah ke bentuk numerik. Hasil yang ditampilkan pada X.head() menunjukkan bahwa seluruh data kategorikal (Sex, BP, Cholesterol) kini sudah berbentuk angka, sehingga siap digunakan pada algoritma machine learning yang memerlukan input data numerik.

- Membuat dan Melatih Model SVM

```
[27]
✓ Os
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Membuat model SVM dengan kernel linear
model = SVC(kernel='linear')
model.fit(X_train, y_train)
```

SVC

SVC(kernel='linear')

Kode ini digunakan untuk membuat dan melatih model Support Vector Machine (SVM) dengan kernel linear menggunakan library scikit-learn. Data X dan y dibagi terlebih dahulu menjadi data latih dan data uji dengan fungsi `train_test_split` (80% data untuk pelatihan dan 20% untuk pengujian). Selanjutnya, model SVM dengan kernel linear dibuat menggunakan `SVC(kernel='linear')` dan dilatih pada data latih menggunakan `model.fit(X_train, y_train)`. Hasil output menunjukkan bahwa model sudah berhasil dibuat dan siap digunakan untuk proses prediksi atau evaluasi selanjutnya.

- Evaluasi Model SVM (Akurasi dan Laporan Klasifikasi)

```
[28]
✓ Os
y_pred = model.predict(X_test)
# Akurasi
print(f"Akurasi: {accuracy_score(y_test, y_pred) * 100:.2f}%")
# Laporan klasifikasi
print("\nLaporan Klasifikasi:\n", classification_report(y_test, y_pred))
```

Akurasi: 100.00%

Laporan Klasifikasi:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	15
1	1.00	1.00	1.00	6
2	1.00	1.00	1.00	3
3	1.00	1.00	1.00	5
4	1.00	1.00	1.00	11
accuracy			1.00	40
macro avg	1.00	1.00	1.00	40
weighted avg	1.00	1.00	1.00	40

Kode ini digunakan untuk mengevaluasi performa model SVM dengan melakukan prediksi pada data uji (`X_test`), kemudian menghitung akurasi dan menampilkan laporan klasifikasi. Akurasi dihitung dengan `accuracy_score` dan ditampilkan dalam persentase, sedangkan `classification_report` menunjukkan metrik precision, recall, dan f1-score untuk masing-masing kelas. Output menunjukkan akurasi

sebesar 100% pada data uji, yang berarti semua prediksi benar, serta nilai precision, recall, dan f1-score sebesar 1.00 pada setiap kelas. Ini menandakan bahwa model sangat baik dalam mengklasifikasikan data pada kasus ini.

- Membuat dan Menampilkan Confusion Matrix

```
[29]
✓ 1s from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
import seaborn as sns

print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

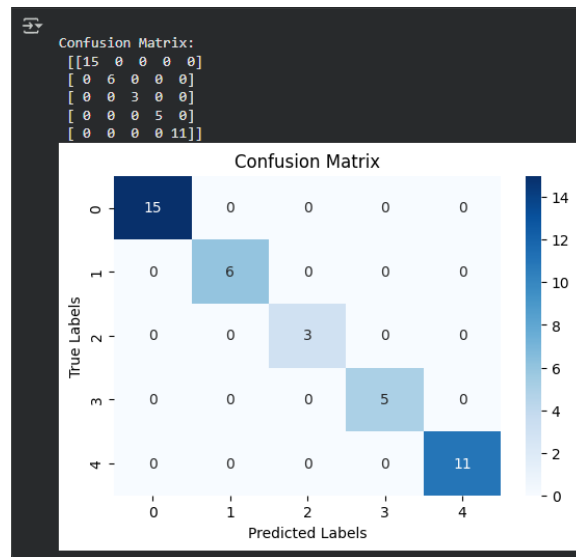
# Buat confusion matrix
cm = confusion_matrix(y_test, y_pred)

# Jika kamu tahu nama kelas (opsional, agar lebih informatif)
# misalnya: class_names = ['Negatif', 'Positif']
# maka tambahkan ke heatmap di bagian "xticklabels" dan "yticklabels"

plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix")
plt.xlabel("Predicted Labels")
plt.ylabel("True Labels")
plt.show()
```

Kode ini digunakan untuk membuat confusion matrix yang berfungsi mengevaluasi performa klasifikasi model dengan melihat jumlah prediksi benar dan salah pada masing-masing kelas. Fungsi `confusion_matrix` dari `sklearn.metrics` digunakan untuk menghitung confusion matrix berdasarkan hasil prediksi dan label asli. Selanjutnya, confusion matrix divisualisasikan dalam bentuk heatmap menggunakan `seaborn` dan `matplotlib` agar interpretasi hasilnya lebih mudah dan informatif. Visualisasi ini menampilkan jumlah prediksi pada setiap kombinasi kelas asli dan kelas prediksi, sehingga dapat membantu mengidentifikasi potensi error atau dominasi kelas tertentu dalam model.

- Output Confusion Matrix



Output ini menampilkan hasil confusion matrix dalam bentuk matriks dan visualisasi heatmap. Angka-angka pada matriks menunjukkan jumlah prediksi yang benar untuk setiap kelas (ditandai pada diagonal) dan jumlah prediksi salah (jika ada di luar diagonal). Tampak seluruh nilai berada pada diagonal, seperti , yang menandakan semua sampel diuji berhasil diprediksi dengan benar sesuai kelas aslinya tanpa kesalahan klasifikasi. Visualisasi heatmap semakin memperjelas hasil tersebut, di mana semua kotak dengan jumlah terisi berada di diagonal utama. Hasil ini menunjukkan performa model sangat baik pada data uji.

- Membuat Scatter Plot Na_to_K vs Age dengan Warna Berdasarkan Drug

```
[30]
✓ Os
# Scatter plot
plt.figure(figsize=(8,6))
# Get the encoded values for coloring
encoded_drugs, drug_labels = pd.factorize(df['Drug'])
scatter = plt.scatter(df['Na_to_K'], df['Age'], c=encoded_drugs, cmap='viridis')

# Label dan judul
plt.xlabel('Na_to_K Ratio')
plt.ylabel('Age')
plt.title('Visualisasi Dataset Drug200: Na_to_K vs Age')

# Create legend handles and labels manually
legend_handles = scatter.legend_elements()[0]
# Map encoded values back to original drug names for legend labels
unique_encoded_drugs = sorted(list(set(encoded_drugs)))
legend_labels = [drug_labels[i] for i in unique_encoded_drugs]

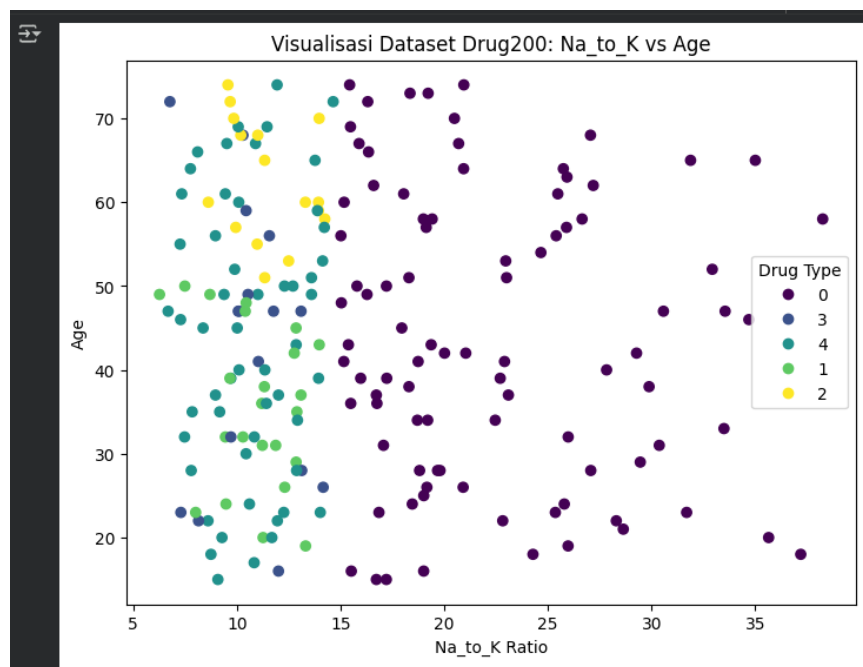
plt.legend(handles=legend_handles, labels=legend_labels, title="Drug Type")

# Tampilkan plot
plt.show()
```

Kode ini digunakan untuk membuat visualisasi scatter plot yang menggambarkan hubungan antara rasio Na_to_K dan usia (Age) pada dataset Drug200.

Warna titik pada plot merepresentasikan kelas obat (Drug) masing-masing data, yang sudah diencoding menjadi numerik. Scatter plot ini memudahkan pengguna untuk melihat pola distribusi data dan kemungkinan perbedaan karakteristik setiap jenis obat berdasarkan dua fitur tersebut. Legenda ditambahkan secara manual sehingga nama asli obat tetap terlihat, dan dengan `cmap='viridis'`, visualisasi menjadi lebih mudah diinterpretasikan menurut tipe obat setiap data.

- Output Scatter Plot Na_to_K vs Age



Output ini merupakan visualisasi scatter plot yang menampilkan hubungan antara variabel Na_to_K Ratio (sumbu X) dan Age (sumbu Y) pada dataset Drug200, dengan warna titik merepresentasikan jenis obat (Drug Type) yang di-encode menggunakan angka. Setiap warna pada plot menunjukkan kategori obat yang berbeda, sehingga pola distribusi tiap tipe obat terhadap dua fitur tersebut bisa diamati dengan jelas. Legenda di sisi kanan memudahkan identifikasi tipe obat berdasarkan warnanya, membantu dalam eksplorasi visual data dan pola karakteristik antar kelas obat.

- Membuat 3D Scatter Plot Klasifikasi Obat

```
[32]
✓ Os
from sklearn.preprocessing import LabelEncoder
from mpl_toolkits.mplot3d import Axes3D

# Encode kolom kategori
le_drug = LabelEncoder()
df['DrugEncoded'] = le_drug.fit_transform(df['Drug'])

le_bp = LabelEncoder()
df['BPEncoded'] = le_bp.fit_transform(df['BP'])

# Siapkan figure 3D
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

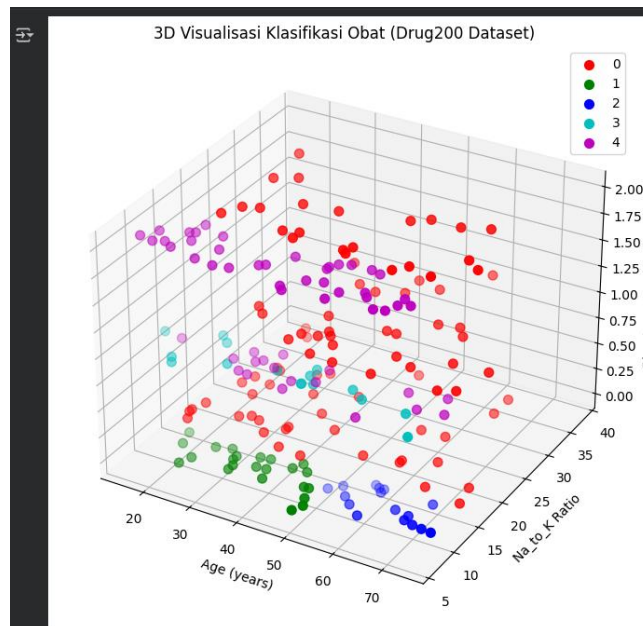
# Warna untuk tiap jenis obat
colors = ['r', 'g', 'b', 'c', 'm']
labels = le_drug.classes_

# Plot tiap jenis obat
for i, drug in enumerate(labels):
    subset = df[df['DrugEncoded'] == i]
    ax.scatter(
        subset['Age'],
        subset['Na_to_K'],
        subset['BPEncoded'],
        color=colors[i % len(colors)],
        label=drug,
        s=50
    )

# Label dan judul
ax.set_xlabel('Age (years)')
ax.set_ylabel('Na_to_K Ratio')
ax.set_zlabel('Blood Pressure (encoded)')
ax.set_title('3D Visualisasi Klasifikasi Obat (Drug200 Dataset)')
ax.legend()
plt.show()
```

Kode ini digunakan untuk membuat visualisasi scatter plot 3D berdasarkan tiga fitur: usia (Age), rasio Na_to_K, dan tekanan darah (BP, yang telah diencoding) pada dataset Drug200. Setiap jenis obat diwakili warna berbeda, dan LabelEncoder digunakan untuk mengubah data kategorikal menjadi numerik. Data tiap kelas obat diplot secara terpisah sehingga distribusi dan pemisahan antar kelas obat dapat diamati dalam ruang tiga dimensi. Visualisasi ini sangat bermanfaat dalam mengeksplorasi pola dan hubungan antara fitur serta efektivitas pemisahan tiap tipe obat pada dataset.

- Output 3D Scatter Plot Klasifikasi Obat



Output ini adalah visualisasi scatter plot 3D yang menggambarkan penyebaran data pada tiga variabel utama: Age (umur), Na_to_K Ratio (rasio sodium terhadap kalium), dan Blood Pressure (BP, dalam bentuk encoded). Setiap jenis obat direpresentasikan oleh warna berbeda pada grafik, sesuai dengan hasil encoding, dan legenda pada pojok kanan atas memudahkan pengenalan tipe obat. Visualisasi ini sangat membantu untuk melihat pola, pemisahan antar kelas obat, serta hubungan multidimensi antar fitur pada dataset Drug200. Data dari setiap kelompok obat tampak terdistribusi dengan pola dan kelompok warna tersendiri.

- Kesimpulan

Praktikum mandiri klasifikasi obat dengan machine learning ini menunjukkan pentingnya proses pra-pemrosesan data, transformasi fitur, dan eksplorasi visualisasi untuk memahami pola data secara menyeluruh. Model SVM dengan kernel linear yang diterapkan pada dataset Drug200 terbukti sangat efektif dan menghasilkan akurasi prediksi hingga 100% pada data uji, dengan hasil evaluasi dan visualisasi yang mendukung pemisahan kelas obat dengan baik. Seluruh rangkaian analisis memperkuat pemahaman bahwa tahapan encoding, pemilihan fitur, serta evaluasi model sangat krusial dalam menghasilkan sistem klasifikasi obat yang akurat dan informatif.

Link dataset Kaggle:

<https://www.kaggle.com/datasets/prathamtripathi/drug-classification>

Link Praktikum 06:

https://github.com/HisyamWildan/TI03_HisyamW.A_0110222206/blob/main/Pertemuan%206/Praktikum%20%26%20Tugas%20Mandiri%2006/notebooks/praktikum06.ipynb

Tugas Mandiri 06:

https://github.com/HisyamWildan/TI03_HisyamW.A_0110222206/blob/main/Pertemuan%206/Praktikum%20%26%20Tugas%20Mandiri%2006/notebooks/latihan06.ipynb